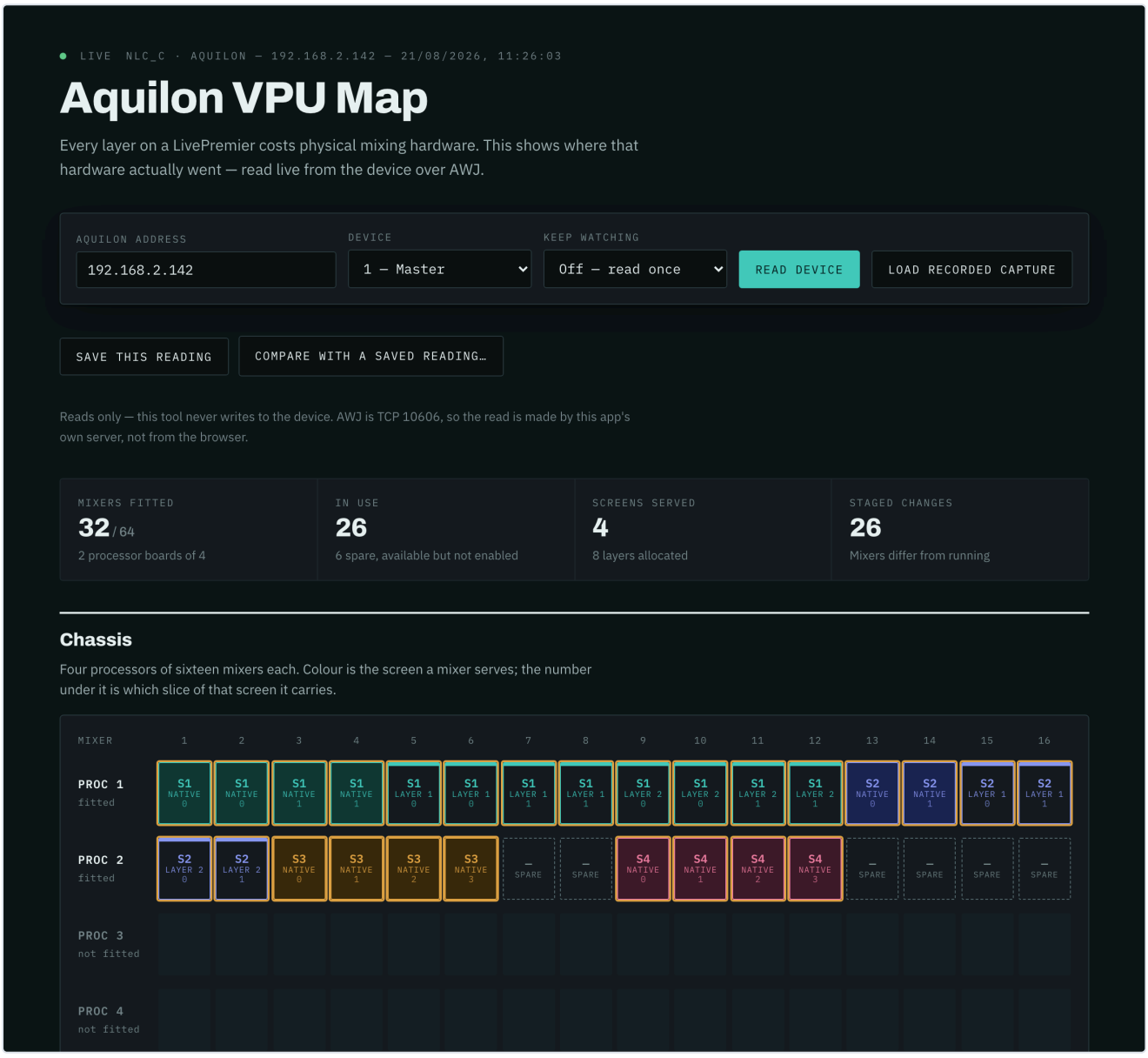
Aquilon VPU Map user guide

Aquilon VPU Map shows **how an Analog Way LivePremier allocates its VPU mixers** across screens, layers and slices, read live from the device or from a recorded capture.

Every layer on a LivePremier costs physical mixing hardware. The device knows where every bit of it went and will happily tell you, but the stock Web RCS only shows it a panel at a time. This puts the whole chassis on one screen.



> **Before you rely on this:** it **has read a real Aquilon C end to end**, in three different configurations. Three of the four recorded captures and the screenshots come from that box, and the second configuration *corrected* two things the first had made look settled. The fourth capture comes from a second Aquilon C, which agreed with the model on 2026-09-09.
>
> Still untested: **Link setups** (devices 2–4), capacities other than 4K and 5K, combined VPUs and **Cut & Fill**. **Optimized mode** has been read once, in the third capture (a screen of three layers over three outputs reporting `isOptimized`), and the tests run off it. There's no frame here between visits, so the untested ones are configurations somebody with hardware has to build.
>
> This codebase was created with AI assistance, directed and reviewed by a human author.

It reads only. Nothing in this tool writes to the device.

What a VPU mixer is

A **VPU mixer** is the physical mixing and scaling resource the device allocates to a *(screen, layer)* pair. An Aquilon has up to four processors of sixteen mixers, **64 in total**, though most chassis are part-populated.

A layer too wide for a single mixer is split across several, each carrying one **slice**. That's why an eight-slice native layer can eat a whole processor board on its own, and why counting layers never tells you whether a configuration will fit.

The device reports this map read-only and keeps two copies: `current` (running) and `new` (staged). This tool shows the running map and **highlights anything the staged one would change**, which is the view you want before you press anything on the desk.

Using it

The desktop app is the easiest way in: double-click it, nothing to install. macOS, Windows or Linux, about 2 MB, because it uses the system WebView rather than bundling a browser.

> The macOS builds are signed and notarised and open normally. The Windows builds are unsigned, so SmartScreen warns once.

Or run the server with `npm start` and open http://localhost:8531.

It's the same UI either way; the only difference is how it reaches the switcher. On the desktop, the connection is made directly rather than through a server.

Screens are named the way you named them, "S1 · Main LED", read from the device. Those names only appear live; the recorded captures are redacted.

Reading the link grid



The manual draws a VPU as an **8×8 field of links**: eight layer links in from the left, eight output links in through the top and out through the bottom. It's a crosspoint field, and this view follows the manual's own figures, read top to bottom the way an output link runs through it:

- **A row is one layer-capacity link, and it carries one layer.** Two layers never share a row. A layer is as tall as its capacity: dual link (up to 4K30) is 1, 4K60 is 2, 5K60 is 4.
- **Columns belong to screens.** Each screen owns a contiguous run of output links, as wide as the number of outputs it uses, and screens sit side by side, so two four-output screens fill a VPU as links 1–4 and 5–8. That means all of a screen's layers start at the same link.
- **A layer's bar is continuous, and breaks at the centre line.** A layer spread over more than four output links takes another layer link and wraps onto it, drawn with the manual's hook. A twelve-output screen's layer costs three links, over two VPUs.
- **Optimized mode lifts that boundary for capacity-2 layers, and only those**, so on an optimized VPU their bars run unbroken across the centre line.
- **A screen's native background isn't layer capacity.** It's reported like a layer and holds mixers, but it's drawn dimmed in a band of its own, **above** the field, and left out of the layer-link count. The native is the bottom of the stack, so it's the first thing on the output link, with layer 1 over it and the rest down the field.
- **A screen that runs out of mixers carries on in the next VPU.** Its next layer is allocated there on the *same* output links, so the link runs out of the bottom of one VPU and into the top of the next. Those two cards are **stacked**, the screen keeps the same columns on both, and the arrows at both ends take the screen's colour, with `↓ VPU 2` under the leaving ones and "continues from VPU 1" on the arriving card.
- **The header names each link.** The screen bar is always there. On a live read, two rows under it say which **region** and which **output plug** each link is (`R1`, `Out 5`), with the output's label, connector type and card in the tooltip. A 4K output is two links wide. Of the recorded captures only the fourth carries the outputs; on the other three the header stops at the screen.

Why the columns are not what the device's keys say

The device reports each mixer as a pair, and **the two halves disagree**.

The **key** is the VPU pipe the mixer is wired to, and those are **interleaved**: a six-output screen's first mixer sits on pipes 1, 3, 5 and 7. The **value** is which of the *screen's* output links that pipe carries, and those are 1, 2, 3, 4: in order, contiguous, and what the manual draws.

Read the keys as columns and you get a screen's layers on scattered links and a bar reaching across the centre line, which the hardware can't do. So the view uses the values.

Nothing in the protocol names the layer link (the row) at all. It doesn't need to: the rules above fix how many links each layer spends and forbid sharing, so the only thing the tool gets to choose is the order down the field, and it follows the device's own mixer allocation order.

Nothing names the output behind a link either. The header takes that from the outputs themselves (each one says which screen and region it's in, and its capacity), dealt out over the screen's links in **output-number order**. The paths and the sums hold on a real box. The order is the only one the object model offers, and no box has confirmed it yet, because every screen read so far had a

single output. So the tool checks the outputs against the screen's own output count and capability figures, and draws no header for a screen that doesn't add up.

Working without a device

Four recorded configurations come with it, and they're **the whole ground truth**. Every one is offered in the app's **Recorded capture** picker, and the tests run off them.

```
node scripts/capture-config.mjs --report
```

audits them with no device needed: what each capture proves, and which questions none of them can answer (a capacity-1 layer, an over-budget configuration, Cut & Fill actually enabled, a screen too wide for one VPU, the order of a screen's links over several outputs).

If you've got hardware and want to settle one of those, CAPTURE-GUIDE.md assumes nothing: what to set up, what to run, and what it settles. Recording one is:

```
node scripts/capture-config.mjs <ip> --name capacity-1 --label "Dual-link layer"
```

which writes a **redacted** capture and lists it in the picker.

If something looks wrong

A layer's bar crosses the centre line and I'm not in Optimized mode. That should be impossible. Either the capacity is 2 and the VPU is optimized, or it's a bug worth reporting with a capture.

A screen's layers are on scattered links. Also a bug. See the columns note above: that's the symptom you get from reading the pipe keys instead of the link values.

The map doesn't match what I just changed on the desk. The tool shows the **running** map. A staged change is highlighted rather than applied; that's what the two copies are there for.

Nothing is reported for a Link setup. Devices 2–4 have never been tested. That's on the list above, not a fault to chase.
